

Cross-Platform Evaluation of SVM and XGBoost Ensemble for Phishing Detection

Christian James C. Cahilig¹

Bachelor in Computer Science

University of Mindanao

Matina, Davao City, 8000

c.cahilig.544797@umindanao.edu.ph

Fionnah Keiz P. Coyoca²

Bachelor in Computer Science

University of Mindanao

Matina, Davao City, 8000

f.coyoca.545499@umindanao.edu.ph

ABSTRACT

Phishing attacks have evolved from email-only threats into multi-channel campaigns targeting users across email and SMS platforms, yet most detection systems remain platform-specific and lack cross-platform evaluation. This study addresses this gap by conducting a cross-platform evaluation of machine learning models for phishing detection, with emphasis on a weighted soft-voting ensemble combining Support Vector Machine (SVM) and XGBoost classifiers. Using publicly available email (82,486 samples) and SMS (5,971 samples) datasets from Kaggle and Mendeley Data, standardized preprocessing and TF-IDF feature extraction were applied to ensure consistent benchmarking. Models including Random Forest, SVM, Neural Network, XGBoost, and the proposed ensemble were evaluated using accuracy, precision, recall, F1-score, ROC-AUC, false positive rate (FPR), false negative rate (FNR), and computational efficiency. Results revealed platform-specific performance variations: Random Forest excelled on email (0.9864 accuracy, 0.0129 FNR), while the SVM+XGBoost ensemble achieved the highest performance on SMS (0.9841 accuracy, 0.9575 F1-score) and demonstrated the most balanced cross-platform profile. The ensemble successfully combined SVM's precision with XGBoost's recall, mitigating individual classifier weaknesses. These findings provide evidence-based recommendations for educational institutions deploying phishing detection systems across diverse communication channels, contributing to more adaptable and security-aware cybersecurity infrastructures. The study supports Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure) by advancing machine learning applications for critical security challenges.

Categories and Subject Descriptors

CCS → Security and privacy → Intrusion/anomaly detection and malware mitigation → Social engineering attacks → Phishing

General Terms

Algorithms; Security; Experimentation; Performance; Design

Keywords

XGBoost; Phishing Detection; Cross-Platform Security; Email and SMS Security; Machine Learning.

1. INTRODUCTION

1.1 Background of the Study

Phishing attacks have evolved from poorly crafted mass emails into sophisticated multi-channel threats targeting users across email, Short Message Service (SMS), and messaging platforms. Attackers now employ advanced social engineering tactics, personalized lures, and multi-stage delivery mechanisms that allow them to bypass traditional rule-based filters [1]. Because these tactics are increasingly effective at evading standard defenses, there is a clear need to shift toward intelligent, behavior-aware detection systems that analyze structural and textual features to identify malicious intent rather than merely searching for known malicious keywords [2].

Researchers have turned to machine learning to address this challenge. A recent comprehensive study by Latif and Pervaiz [3] specifically examined phishing email detection using a dataset of 39,000 samples. Their work demonstrated that ensemble learning techniques, particularly XGBoost and Random Forest, significantly outperform other models, achieving accuracy rates as high as 99.00% when combined with preprocessing techniques like feature engineering and SMOTE balancing. This reinforces the importance of advanced classification techniques in phishing detection. Similarly, Ghaleb [4] conducted a comparative study of machine learning approaches for email phishing detection, evaluating multiple algorithms including Random Forest and XGBoost on real email datasets, further confirming the effectiveness of ensemble methods for email security.

The baseline study by Sepillo et al. [2] established that classical classifiers like Random Forest, Support Vector Machine, and Neural Networks achieve solid performance in educational contexts. Their research, conducted within Philippine educational institutions, showed that implementing ML-based systems reduced phishing success rates by 45-60%, decreased compromised accounts by 52%, and increased detection speed by 40% compared to traditional methods. However, this work focused on email-based detection within a single platform, implicitly highlighting the need for cross-platform evaluation and deeper analysis of security-aware metrics like false negative rate.

Single-platform approaches have significant limitations. While Random Forest is valued for robustness and interpretability, it can struggle with the sparse, short-text characteristics of SMS messages. XGBoost delivers high precision but requires careful tuning and computational resources [5]. Jain and Gupta [6] proposed a novel ensemble-based approach for phishing email detection, demonstrating that combining multiple classifiers can achieve superior performance compared to individual models,

though their work remained confined to email platforms. Loh et al. [7] achieved high accuracy (>94%) using CNN deep learning for email phishing, yet did not address computational trade-offs or cross-platform adaptability. More importantly, most existing systems focus primarily on email datasets, leaving SMS-based attacks unaddressed despite their growing prevalence [1].

Table 1 summarizes the comparison of selected recent studies against the proposed work, highlighting persistent research gaps.

Table 1. Related Studies

Ref No.	Language / Dataset	Approach	Results	Gaps Addressed
[2]	English (Email)	Comparative evaluation of Random Forest, SVM, and Neural Networks	Achieved measurable reduction in phishing success rates (45–60%) and improved detection speed	Focused only on email; no SMS evaluation; no ensemble-based probability fusion
[3]	English (Email)	XGBoost and Random Forest with preprocessing and SMOTE	Achieved up to 99% accuracy using ensemble techniques	Limited to email platform; emphasized accuracy over security-aware metrics (FPR/FNR)
[4]	English (Email)	Comparative study of tree-based and boosting algorithms	Confirmed strong performance of ensemble methods for phishing detection	Treated phishing as general classification task; no cross-platform evaluation; limited discussion of asymmetric error costs
[6]	English (Email)	Ensemble machine learning for phishing detection	Demonstrated improved performance over standalone classifiers	No threshold optimization for security-sensitive contexts; confined to email-only detection
[7]	English (Email)	Deep learning (CNN-based detection)	Achieved >94% classification accuracy	Did not evaluate computational efficiency or security-aware metrics; no SMS evaluation

The matrix illustrates a consistent pattern: prior works make significant contributions but remain confined by scope. Latif and Pervaiz [3] provide strong evidence for XGBoost's power in email phishing, achieving up to 99% accuracy with ensemble methods. However, their evaluation focuses on standard metrics and does not explicitly report or analyze FPR and FNR, which are critical for understanding security trade-offs in real-world deployment. Similarly, Ghaleb [4] confirms ensemble method effectiveness for email detection but treats phishing as a standard classification task without emphasizing the asymmetric costs of false negatives versus false positives. Jain and Gupta [6] demonstrate the potential of ensemble approaches and achieve strong accuracy, yet their work similarly lacks explicit FPR/FNR analysis and does not explore threshold optimization to minimize missed attacks. The baseline study [2] stands out by explicitly addressing security metrics within educational contexts, though it remains focused on single-platform evaluation. Loh et al. [7] achieve high accuracy with deep learning but do not address security metrics at all. No existing study systematically evaluates

ensemble approaches like SVM and XGBoost across both email and SMS platforms under standardized conditions while also prioritizing false negative reduction, false positive control, and computational efficiency.

Therefore, this study extends the original work by introducing a cross-platform evaluation framework with a weighted soft-voting ensemble model combining SVM and XGBoost, benchmarked against traditional models across both email and SMS phishing datasets. This study seeks to determine whether ensemble approaches leverage the complementary strengths of different algorithms to enable superior and more consistent performance across communication channels compared to single-model baselines. By conducting systematic cross-platform evaluation under standardized preprocessing conditions, this study investigates whether a weighted soft-voting ensemble can provide more consistent performance across heterogeneous platforms. The findings aim to provide educational institutions with evidence-based recommendations for deploying unified, robust phishing detection defenses.

1.2 Purpose and Description

The purpose of this study is to evaluate and enhance phishing detection performance across multiple communication channels, specifically email and SMS, using a weighted soft-voting ensemble combining Support Vector Machine and XGBoost as the primary classification framework. As phishing attacks become increasingly cross-platform, there is a need for detection systems capable of adapting to diverse textual characteristics, including high-dimensional email data and short, sparse SMS messages. This study contributes to the field of cybersecurity by examining whether combining linear margin-based classification with nonlinear boosting mechanisms provides measurable improvements over standalone models.

The study supports secure digital infrastructures in educational institutions and aligns with Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure). The findings benefit academic institutions, IT administrators, and cybersecurity researchers by providing evidence-based recommendations on model selection for phishing detection.

This study implements baseline models (Random Forest, SVM, and Neural Networks) alongside XGBoost and compares them with the proposed SVM and XGBoost ensemble using standardized preprocessing and TF-IDF feature extraction. Performance is evaluated using classification metrics (accuracy, precision, recall, F1-score, ROC-AUC), security-aware metrics (FPR, FNR), and computational efficiency (training and prediction time). The system is implemented in Python using Scikit-learn and XGBoost libraries within a Google Colab environment.

1.3 Objectives

1.3.1 General Objective

To evaluate phishing detection across email and SMS platforms using a weighted soft-voting ensemble framework combining Support Vector Machine and XGBoost classifiers.

1.3.2 Specific Objectives

1.3.2.1 To assess the performance of traditional machine learning models (Random Forest, Support Vector Machine, and Neural Networks) on both email and SMS phishing datasets.

1.3.2.2 To evaluate the performance of individual base classifiers (SVM and XGBoost) relative to traditional machine learning models across both email and SMS domains.

1.3.2.3 To analyze the effectiveness of an ensemble model, particularly SVM and XGBoost combination, in improving cross-platform phishing detection.

1.3.2.4 To analyze the impact of model choice on security-aware metrics, with emphasis on achieving an optimal balance between precision and recall through the F1-score.

1.4 Scope and Limitation

This study focuses on evaluating machine learning models for phishing detection using publicly available email and SMS datasets obtained from Kaggle [8] and Mendeley Data [9]. The study compares baseline models (Random Forest, SVM, Neural Networks) and XGBoost with an ensemble configuration of SVM and XGBoost, across two communication channels. Standardized preprocessing, TF-IDF feature extraction, and structural feature engineering are applied to ensure consistent evaluation.

The system is implemented in Python using Google Colab and Scikit-learn/XGBoost libraries. The primary beneficiaries are educational institutions seeking improved phishing detection strategies. The study operates as an experimental prototype framework rather than a deployed enterprise solution.

The study is limited to textual content analysis and does not evaluate multimedia phishing attacks, voice phishing (vishing), or URL behavioral analysis. It relies on secondary datasets, which may not perfectly represent real-world attack distributions. Additionally, results depend on dataset characteristics, preprocessing configurations, and class imbalance handling.

2. METHODS

2.1 Data Gathering

This study utilized two publicly available secondary datasets to evaluate phishing detection across email and SMS platforms. Secondary data sources were selected to ensure reproducibility and enable benchmarking against existing studies.

2.1.1 Email Dataset

The phishing email dataset was obtained from Kaggle, contributed by Al-Am [8]. This dataset contains labeled email samples comprising both phishing and legitimate messages. The dataset includes a combined textual field (text_combined) used for natural language processing. The dataset consists of approximately 82,500 samples, making it suitable for evaluating model scalability in high-dimensional environments.

2.1.2 SMS Dataset

The SMS phishing dataset was sourced from Mendeley Data, provided by Mishra and Soni [9]. This dataset contains SMS messages with binary labels indicating whether each message is

phishing (smishing) or legitimate (ham). The dataset includes pre-extracted metadata features including URL, email address, and phone number presence, and the raw message text, making it valuable for hybrid feature engineering approaches.

2.1.3 Data Upload and Initialization

Both datasets were uploaded to Google Colab using the files.upload() method. The email dataset was loaded as phishing_email.csv and the SMS dataset as Dataset_5971.csv. Data inspection was performed using pandas info() and head() methods to verify structure and identify data quality issues.

2.2 Dataset Analysis

Prior to preprocessing and model development, data analysis was conducted to examine dataset composition, class distribution, textual characteristics, and structural properties. This is essential to understand the statistical behavior of the data and to justify the subsequent preprocessing and feature engineering strategies.

2.2.1 Email Dataset

The email dataset consists of 82,486 instances, each containing a combined textual field (text_combined) and a binary class label (label), where 0 represents legitimate emails and 1 represents phishing emails. Inspection of the dataset confirmed that no missing values were present in either column, indicating complete data availability for analysis.

Table 2. Email Class Distribution

label	No. of Rows	Count Percentage (%)
1	42891	52.0
0	39595	48.0

Class distribution analysis revealed that 42,891 emails (52.0%) were labeled as phishing, while 39,595 emails (48.0%) were labeled as legitimate. This near-balanced distribution reduces the severity of class imbalance concerns and supports the use of standard stratified splitting procedures during model training.

To further understand textual variability, character-level text length statistics were computed for the raw email corpus. The average email length was 1,288.75 characters, with a median of 558 characters and a standard deviation of 15,496.75. The minimum length observed was 1 character, while the maximum length reached 4,279,526 characters. The extremely high maximum value indicates the presence of unusually long messages, likely containing forwarded content, attachments encoded as text, or extensive logs. The large standard deviation confirms substantial dispersion and suggests a heavy-tailed distribution. Interquartile statistics show that 50% of emails fall between 276 and 1,338 characters, indicating that the majority of emails are moderate in length while a small subset significantly increases overall variance.

Table 3. Email Text Length

Email Text Length	
count	82,486.00
mean	1,288.751
std	15,496.75
min	1
25%	276.0000
50%	558.0000
75%	1,338.000
max	4,279,526

Vocabulary analysis of the raw emails identified 626,543 unique tokens. This high dimensionality reflects case sensitivity, punctuation artifacts, numeric strings, URLs, and noise typical of unprocessed textual data. Such vocabulary size would substantially increase sparsity in vector representations and negatively affect computational efficiency, thereby motivating systematic preprocessing.

2.2.2 SMS Dataset

The SMS dataset contains 5,971 messages with associated metadata fields, including indicators for URL presence, email presence, and phone number presence. Binary normalization of the label column produced 4,844 legitimate messages (81.13%) and 1,127 smishing messages (18.87%).

Table 4. SMS Class Distribution

label	No. of Rows	Count Percentage (%)
0	4844	81.13
1	1127	18.87

Unlike the email dataset, the SMS dataset exhibits noticeable class imbalance, with legitimate messages dominating the corpus. This imbalance justifies the use of class-weighted learning and scale adjustment in certain models. Moreover, no missing values were detected across any SMS dataset columns, confirming structural completeness.

Text length analysis revealed that SMS messages are significantly shorter than emails. The mean character length was 83.24, with a median of 66 characters and a standard deviation of 58.68. The shortest message contained 2 characters, while the longest reached 910 characters. The interquartile range shows that 50% of SMS messages fall between 38 and 127 characters. This relatively compact distribution reflects the inherent brevity of SMS communication.

Table 5. SMS Text Length

SMS Text Length	
count	5971.000000
mean	83.239826
std	58.681945
min	2.000000
25%	38.000000
50%	66.000000
75%	127.000000
max	910.000000

Metadata feature inspection showed zero variance in the has_url, has_email, and has_phone indicators in their encoded form. This suggests that these explicit flags do not contribute discriminative power in the dataset’s current representation, and that structural cues must instead be inferred from textual content itself. Still, these features were retained for completeness.

2.3 Data Preprocessing

Following exploratory analysis, a structured preprocessing pipeline was applied to normalize textual content, reduce noise, and prepare the data for feature extraction.

2.3.1 Text Normalization and Cleaning

The first stage of preprocessing involved text normalization. All text was converted to lowercase to eliminate case-based token duplication. URLs and web links were removed using regular expressions to reduce sparsity introduced by unique link strings. HTML tags were eliminated to remove formatting artifacts.

Punctuation symbols and numeric characters were stripped to focus on lexical content. Stopwords were removed using the NLTK English stopwords list to eliminate high-frequency function words that contribute limited semantic value.

Finally, lemmatization was applied using WordNet to reduce inflected words to their base forms. The effect of this cleaning process was evaluated by comparing text length statistics before and after preprocess

For the email dataset, the mean text length decreased from 1,288.75 characters to 1,085.55 characters. The median length decreased from 558 to 483 characters. The maximum length was reduced from 4,279,526 to 3,518,258 characters.

These reductions confirm the removal of extraneous tokens such as numeric sequences, URLs, and formatting noise. The standard deviation also decreased from 15,496.75 to 12,863.67, indicating reduced dispersion after cleaning.

Table 6. Email Raw vs Cleaned Text Length Comparison

	Raw Length	Cleaned Length
count	82,486.00	82,486.0
mean	1,288.751	1,085.550
std	15,496.75	12,863.67
min	1.000000	0.000000
25%	276.0000	232.0000
50%	558.0000	483.0000
75%	1,338.000	1,106.000
max	4,279,526	3,518,258

Vocabulary size analysis further illustrates the impact of normalization. The number of unique tokens decreased from 626,543 to 450,241, representing approximately a 28% reduction in vocabulary dimensionality. Although the cleaned vocabulary size remained high, TF-IDF dimensionality was constrained to 10,000 (Email) and 5,000 (SMS) features to ensure computational efficiency.

Table 7. Email Vocabulary Size Comparison

	Raw Vocabulary	Cleaned Vocabulary
Size	626543	450241

Nonetheless, this reduction improves computational efficiency in TF-IDF vectorization and decreases sparsity without discarding semantic meaning.

Table 8. SMS Raw vs Cleaned Text Length Comparison

	Raw Length	Cleaned Length
count	5971.000000	5971.000000
mean	83.239826	51.939541
std	58.681945	37.3810014
min	2.000000	0.000000
25%	38.000000	23.000000
50%	66.000000	41.000000
75%	127.000000	77.000000
max	910.000000	513.000000

For the SMS dataset, preprocessing produced proportionally greater compression due to the shorter nature of messages. The mean text length decreased from 83.24 to 51.94 characters, and the median decreased from 66 to 41 characters. The maximum length was reduced from 910 to 513 characters. These reductions

indicate successful elimination of non-essential elements such as punctuation, stopwords, and numeric artifacts, resulting in more compact lexical representations.

Visual inspection of raw versus cleaned text length distributions confirms that preprocessing shifts the distributions leftward while preserving relative structure between classes.

2.3.2 Feature Engineering

Feature engineering extracts meaningful attributes from raw data to improve model performance. In phishing detection, relying solely on textual content may overlook behavioral indicators such as capitalization, punctuation, or embedded links. Therefore, the study incorporates structural (email) and metadata (SMS) features alongside textual representations.

For the email dataset, structural features such as the number of URLs, number of exclamation marks, number of uppercase characters, number of digits, and overall text length were extracted. Such indicators reflect urgency tactics, formatting emphasis, and embedded links commonly used in phishing communications.

For the SMS dataset, structural attributes included URL presence, email presence, phone number presence, character length, and uppercase ratio. While explicit URL and contact indicators showed small variation, text length and uppercase ratio provided additional stylistic cues that complement lexical features.

The integration of structural attributes with textual features enables the models to capture both semantic and behavioral characteristics.

2.3.3 TF-IDF Feature Representation

Following cleaning and feature engineering, textual data were transformed using the Term Frequency-Inverse Document Frequency (TF-IDF) representation. It is widely used in text classification because it reduces the weight of common words, emphasizes terms indicative of phishing, produces numeric vectors suitable for machine learning models, and overall controls sparsity and improves computational efficiency.

For the email dataset, a maximum of 10,000 features was used with unigram and bigram combinations and document frequency thresholds to reduce noise. For the SMS dataset, the feature space was limited to 5,000 features with similar n-gram configurations.

The final feature matrices were constructed by concatenating TF-IDF vectors with engineered structural features using sparse matrix stacking. This hybrid representation allows machine learning models to leverage both lexical semantics and structural signals during classification.

2.3.4 Data Splitting

Lastly, both the Email and SMS datasets were divided into training and testing sets using a stratified 80:20 split. Stratification was applied to preserve the original class distribution of phishing and legitimate messages in both subsets.

This approach ensures that minority phishing instances remain proportionally represented during both training and evaluation.

All preprocessing steps, including TF-IDF vectorization, were fitted exclusively on the training data and subsequently applied to the testing data to prevent information leakage. The testing set was strictly reserved for final performance evaluation and was not used during model tuning or optimization. A fixed random seed was applied throughout the splitting and training process to ensure experimental reproducibility.

This structured experimental setup guarantees fair comparison between individual classifiers and the weighted soft-voting ensemble while maintaining methodological integrity.

2.4 Design

This subsection presents the conceptual framework and system architecture of the proposed study.

2.4.1 Conceptual Framework

The conceptual framework of this study is grounded in ensemble learning theory, which suggests that combining multiple classifiers can improve predictive performance when base learners capture different structural characteristics of the data distribution [10]. In phishing detection, individual classifiers often exhibit performance variability across heterogeneous textual sources. Therefore, integrating complementary models may enhance cross-platform adaptability.

Support Vector Machines (SVM) have demonstrated strong performance in high-dimensional text classification tasks due to their margin maximization principle and effectiveness in sparse feature spaces [11][12]. Their ability to construct optimal separating hyperplanes makes them well-suited for TF-IDF-based textual representations commonly used in phishing detection.

Conversely, gradient boosting methods such as XGBoost have shown strong performance in structured and high-dimensional classification tasks due to their ability to model nonlinear feature interactions while incorporating regularization to mitigate overfitting [13]. XGBoost constructs decision trees sequentially, where each subsequent tree minimizes the residual errors of the preceding ensemble. This learning strategy enables the model to iteratively refine prediction errors and capture complex structural relationships in phishing-related textual patterns.

Additionally, XGBoost demonstrates robustness in handling feature sparsity and class imbalance, making it well-suited for phishing detection scenarios [12]. To leverage the complementary

strengths of these approaches, the present study adopts a weighted soft-voting ensemble architecture consisting of:

1. Linear Support Vector Machine (SVM), selected for its strong generalization capability in high-dimensional lexical feature spaces.
2. XGBoost classifier, selected for its nonlinear modeling capacity and robustness to structural feature variation.

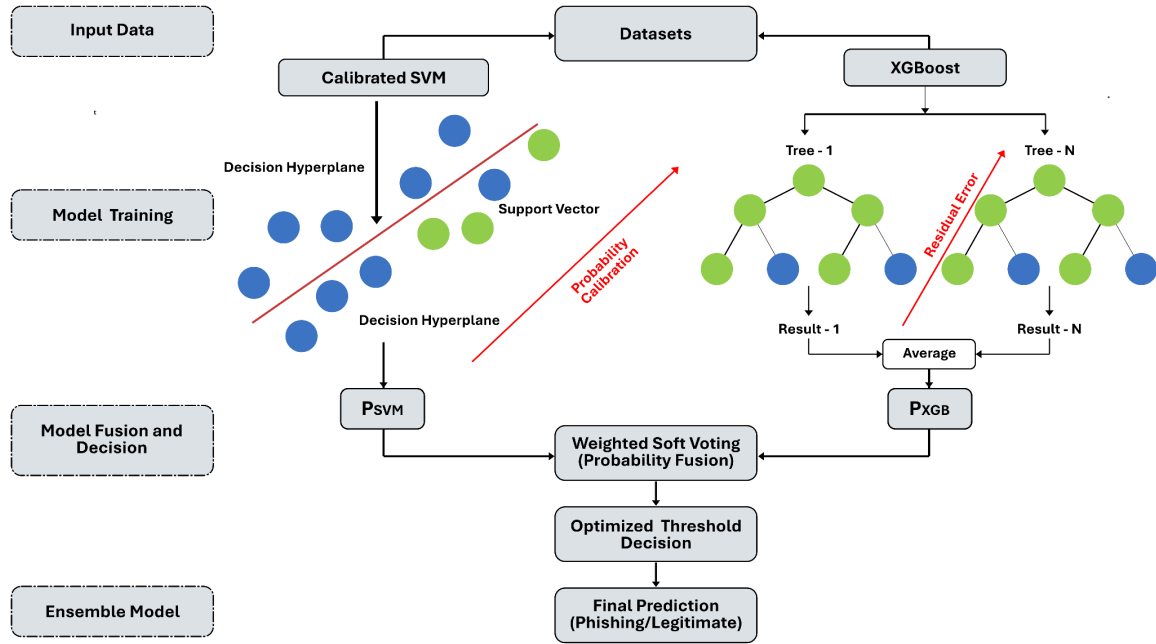


Figure 1. Conceptual Framework of the Ensemble Model

However, since LinearSVC does not inherently produce probabilistic outputs, probabilistic calibration is applied using Platt scaling [14] through cross-validated sigmoid calibration.

This transforms deterministic decision scores into calibrated posterior probabilities suitable for ensemble fusion.

The final probability is computed through weighted soft voting:

$$\mathbf{P}_{\text{final}} = (\mathbf{w}_1 \mathbf{P}_{\text{SVM}} + \mathbf{w}_2 \mathbf{P}_{\text{XGB}}) / (\mathbf{w}_1 + \mathbf{w}_2)$$

where $\mathbf{P}_{\text{SVM}}$ and $\mathbf{P}_{\text{XGB}}$ represent the calibrated probability outputs of the individual classifiers, and $\mathbf{w}_1$ and $\mathbf{w}_2$ denote their respective ensemble weights. The optimal weights are determined through validation-based search to maximize classification performance.

To convert the fused probability into a binary classification decision, an optimized threshold mechanism is applied. Rather than using a fixed 0.5 cutoff, the decision threshold is selected through validation-driven optimization to maximize the F1-score while maintaining an appropriate balance between precision and recall. This is particularly critical in phishing detection contexts, where minimizing false negatives is essential to mitigate security risks.

The overall architecture, illustrated in Fig. 1, presents the sequential flow from raw input data through preprocessing, feature extraction, parallel model training, probabilistic calibration, weighted probability fusion, threshold optimization, and final phishing/legitimate classification output.

2.5 Model Implementation

This section discusses the classification models implemented in the proposed ensemble framework. It covers the justification for selecting each model, their learning mechanisms, and their configuration within the system.

2.5.1 Ensemble Model

The proposed model of this study is a weighted soft-voting ensemble combining a calibrated Linear SVM and an XGBoost classifier. The objective of this model is to integrate the strong margin-based linear separation capability of SVM, and the nonlinear feature interaction modeling of XGBoost.

2.5.1.1 Model Preparation

Prior to ensemble construction, the base classifiers were developed and optimized independently using the training data.

The Linear Support Vector Machine was trained with class-balanced weighting to mitigate class imbalance effects. Probabilistic calibration was applied using Platt scaling implemented through CalibratedClassifierCV with three-fold cross-validation, to account for LinearSVC not generating probability estimates by default. This step transforms decision scores into calibrated probability outputs, ensuring compatibility with the soft-voting mechanism employed in the ensemble framework.

The XGBoost classifier was trained using gradient boosting with tree-based learners. Class imbalance was addressed using the `scale_pos_weight` parameter, computed as the ratio of negative to positive samples within the training set. Hyperparameter tuning was conducted via grid search to determine optimal values for the number of estimators, maximum tree depth, learning rate, subsampling ratio, and column sampling ratio. The final

configuration was selected based on validation performance derived exclusively from the training data.

Both calibrated SVM and optimized XGBoost models were finalized before proceeding to ensemble weight optimization.

2.5.1.2 Weight Optimization via Probability Fusion

To efficiently determine the optimal ensemble weights, a probability-based weight optimization strategy was implemented.

Instead of repeatedly retraining ensemble models, predicted probabilities from the trained SVM and XGBoost models were precomputed. Probabilities were then calculated using weighted soft voting as discussed in Section 2.4.1.

A discrete grid search over weight combinations from values 1–4 was performed to identify the pair that maximized F1-score on the validation set. This vectorized approach significantly reduced computational overhead, especially for large datasets, while yielding results equivalent to exhaustive VotingClassifier retraining.

2.5.1.3 Threshold Optimization

Following weight selection, an additional search was conducted to identify the optimal classification threshold within the range 0.40 - 0.56. This threshold tuning was performed to improve the precision–recall balance, which is critical in phishing detection scenarios where false negatives can have severe consequences.

Moreover, the search range was centered around the default 0.50 decision boundary to balance precision–recall trade-offs while avoiding excessive bias toward recall that could inflate false positives.

2.5.1.4 Final Ensemble Construction

After identifying optimal weights and thresholds, a final soft-voting VotingClassifier was constructed using the selected weights. The ensemble was trained once on the full training dataset. Evaluation was conducted using the optimized threshold. This procedure ensures computational efficiency during optimization, deployment readiness, and consistency across Email and SMS datasets

2.6 Evaluation Metrics

Model performance was evaluated using standard classification metrics, including accuracy, precision, recall, and F1-score. Accuracy reflects the overall correctness of each model, while precision measures the reliability of phishing predictions by indicating how many detected attacks are truly malicious. Recall was treated as a primary metric, as it captures the model's ability to correctly identify phishing attempts and minimize false negatives, which pose significant security risks. The F1-score provides a balanced assessment by combining precision and recall, making it particularly useful for evaluating performance on imbalanced datasets.

In addition, computational efficiency, including training and prediction time, ROC-AUC, and security-aware metrics such as False Positive Rate(FPR) and False Negative Rate(FNR) was also considered for understanding model discrimination ability

and addressing asymmetric classification costs inherent in cybersecurity.

2.7 Model Persistence

Trained models, vectorizers, and optimized thresholds were saved to disk using .pkl files, ensuring reproducibility and supporting future reuse without retraining.

3. RESULTS

3.1 Overview of Experimental Evaluation

This chapter presents the experimental results of the evaluated machine learning models across the Email and SMS phishing datasets. The models assessed include Random Forest (RF), Support Vector Machine (SVM), Neural Network (NN), XGBoost, and the proposed SVM+XGBoost weighted soft-voting ensemble. All experiments were implemented using Python and the Scikit-learn and XGBoost libraries.

Performance was evaluated using classification metrics (Accuracy, Precision, Recall, F1-score, ROC-AUC),

security-aware metrics (False Positive Rate and False Negative Rate), and computational efficiency (training and prediction time).

All results were obtained using the stratified 80:20 train–test split described in Chapter 2.

3.2 Results on the Email Dataset

Random Forest achieved the highest overall performance, with an accuracy of 0.9864 and F1-score of 0.9869. Its ROC-AUC of 0.9987 indicates near-perfect discrimination between phishing and legitimate emails. Most importantly, Random Forest recorded the lowest False Negative Rate (0.0129) among all models, demonstrating strong capability in identifying phishing emails without missing malicious instances.

The proposed SVM+XGBoost ensemble followed closely, achieving an accuracy of 0.9830 and F1-score of 0.9837. While slightly below Random Forest, the ensemble maintained balanced performance across precision and recall metrics. Its False Negative Rate (0.0136) was competitive with the top performer, indicating effective phishing detection.

Table 9. Model Evaluation Matrix Results for Email

Model	Accuracy	Precision	Recall	F1	ROC_AUC	FPR	FNR
Random Forest	0.9864	0.9868	0.9871	0.9869	0.9987	0.0143	0.0129
Ensemble (SVM + XGBoost)	0.9830	0.9811	0.9864	0.9837	0.9984	0.0206	0.0136
Neural Network	0.9820	0.9863	0.9790	0.9826	0.9975	0.0148	0.0210
XGBoost	0.9788	0.9731	0.9865	0.9797	0.9978	0.0295	0.0135
Support Vector Machine	0.9574	0.9971	0.9207	0.9574	0.9978	0.0029	0.0793

Neural Network ranked third, with an F1-score of 0.9826. Its performance was consistent across metrics, though its higher False Negative Rate (0.0210) suggests more missed phishing attempts compared to Random Forest and the ensemble.

XGBoost demonstrated high recall (0.9865), indicating strong ability to detect phishing emails. However, this came at the cost of the highest False Positive Rate (0.0295) among all models, meaning it was more likely to incorrectly classify legitimate emails as phishing.

Support Vector Machine exhibited the most conservative classification behavior. It achieved the highest precision (0.9971) and lowest False Positive Rate (0.0029), rarely misclassifying legitimate emails. However, this conservative approach resulted in the highest False Negative Rate (0.0793), leaving a substantial portion of phishing emails undetected.

3.3 Results on the SMS Dataset

On the SMS dataset, the SVM+XGBoost ensemble achieved the highest overall performance, with an accuracy of 0.9841 and F1-score of 0.9575. Its ROC-AUC of 0.9966 indicates strong classification capability across different threshold values. Rather than the standard 0.5, the ensemble required a slightly adjusted decision threshold (0.53) to optimize the balance between precision and recall. This adjustment helped reduce false positives while maintaining strong phishing detection performance.

Neural Network ranked second, achieving an F1-score of 0.9485. Its competitive performance suggests that nonlinear deep learning models can capture useful structural features even in short-text messages. Support Vector Machine again demonstrated highly conservative behavior, achieving the highest precision

(0.9950) and lowest False Positive Rate (0.0010) among all models. However, this came at the cost of the highest False Negative Rate (0.1150), meaning over 11% of phishing messages went undetected.

XGBoost and Random Forest achieved similar performance, with F1-scores of 0.9364 and 0.9318 respectively. Both models demonstrated reasonable detection capability but fell short of the ensemble's balanced performance.

Table 10. Model Evaluation Matrix Results for SMS

Model	Accuracy	Precision	Recall	F1	ROC_AUC	FPR	FNR
Ensemble (SVM + XGBoost)	0.9841	0.9683	0.9469	0.9575	0.9966	0.0072	0.0531
Neural Network	0.9808	0.9593	0.9381	0.9485	0.9943	0.0093	0.0619
Support Vector Machine	0.9774	0.9950	0.8850	0.9368	0.9949	0.0010	0.1150
XGBoost	0.9766	0.9626	0.9115	0.9364	0.9937	0.0083	0.0885
Random Forest	0.9749	0.9579	0.9071	0.9318	0.9935	0.0093	0.0929

4. DISCUSSION

4.1 Interpretation of Results on the Email Dataset

The experimental results presented in Chapter 3 indicate that Random Forest achieved the highest overall performance on the Email phishing dataset, outperforming all other evaluated models across most classification metrics. As shown in Table 9, Random Forest recorded the highest accuracy, F1-score, and ROC-AUC, while also achieving the lowest False Negative Rate.

The strong performance of Random Forest may be attributed to its ensemble bagging architecture, which constructs multiple decision trees using randomized subsets of features and training samples. This design allows the model to effectively capture complex lexical patterns and feature interactions within the richer TF-IDF feature space of email messages. As discussed in Section 2.2.1, email messages contain richer textual structure and higher feature density, which allows tree-based models like Random Forest to fully exploit complex lexical patterns through their bagging architecture.

Although the proposed SVM+XGBoost ensemble achieved competitive performance (F1-score of 0.9837 and ROC-AUC of 0.9984), it did not surpass Random Forest on this dataset. This suggests that while the ensemble successfully integrates complementary learning strategies, the structural characteristics of the email dataset may already be well captured by the Random Forest classifier alone.

Another notable observation is the behavior of the Support Vector Machine model, which achieved the highest precision (0.9971) and lowest False Positive Rate (0.0029) among all models. This indicates that SVM adopts a more conservative classification boundary, minimizing false alarms but at the cost of higher false negatives. This behavior is reflected in its higher

False Negative Rate (0.0793), which suggests that some phishing emails remain undetected.

From a cybersecurity perspective, false negatives are particularly critical because undetected phishing messages may lead to successful attacks. Therefore, models that maintain a lower FNR while preserving high precision are generally preferable for phishing detection systems.

4.2 Interpretation of Results on the SMS Dataset

In contrast to the Email dataset results, the SVM+XGBoost weighted soft-voting ensemble achieved the highest overall performance on the SMS dataset, as reported in Table 10. The ensemble recorded the highest Accuracy (0.9841), F1-score (0.9575), and ROC-AUC (0.9966), demonstrating its effectiveness in detecting phishing within short-text communication channels.

SMS phishing messages typically contain shorter textual content and limited contextual information, which often results in sparse feature representations after TF-IDF vectorization. This finding directly corresponds to the SMS dataset characteristics described in Section 2.2.2—shorter text length, sparser representations, and greater class imbalance—where ensemble methods provide more robust performance. In such scenarios, relying on a single classifier may limit detection capability because different models capture distinct structural patterns within the data. By integrating the calibrated probability outputs of both classifiers through weighted soft voting, the ensemble can produce more robust predictions than either model individually.

The ensemble also required a slightly adjusted decision threshold (0.53) to optimize the precision-recall balance. This adjustment highlights the flexibility of probabilistic ensemble systems,

which allow threshold tuning to meet the operational priorities of phishing detection systems.

Among the standalone models, the Neural Network demonstrated the second-best performance, suggesting that nonlinear deep learning models may also capture useful structural features in short-text phishing messages. However, the neural network did not surpass the ensemble model in overall performance.

4.3 Comparison Between the Ensemble and Standalone Classifiers

A key objective of this study was to evaluate whether an ensemble of SVM and XGBoost could provide competitive performance relative to individual classifiers across different communication platforms.

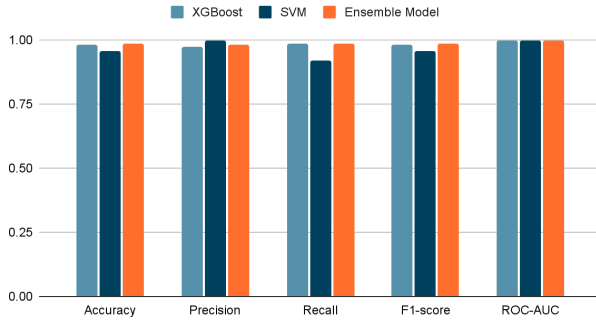


Figure 2. Performance Comparison of the Ensemble Model and its Base Classifiers on the Email Dataset

On email data, Figure 2 reveals distinct strengths for each classifier. SVM achieves the highest precision among the three, reaching approximately 0.997, but its recall drops substantially to around 0.921. XGBoost shows the opposite pattern wherein its recall reaches 0.987, while its precision bar is lower at 0.973. The ensemble occupies a middle ground, with a precision at 0.981 and recall at 0.986, demonstrating its ability to balance the complementary strengths of both base classifiers. For F1-score, the ensemble's bar (0.984) falls between SVM (0.957) and XGBoost (0.980), indicating that the weighted voting mechanism successfully integrates both models' predictions.

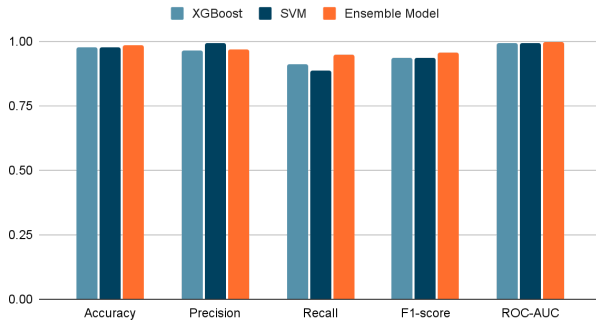


Figure 3. Performance Comparison of the Ensemble Model and its Base Classifiers on the SMS Dataset

On SMS data, the ensemble's balancing effect is even more pronounced. SVM's precision dominates at 0.995, but its recall drops sharply to 0.885. XGBoost improves recall to 0.912 but at the cost of lower precision (0.963). The ensemble showed the most balanced profile with its precision at 0.968 and recall at 0.947, resulting in the highest F1-score (0.957) among the three models. This visual representation clearly demonstrates how the ensemble leverages SVM's strength in avoiding false positives while incorporating XGBoost's superior ability to detect actual phishing messages.

These bar graphs support the theoretical motivation for ensemble learning in that by combining classifiers with different learning biases can improve generalization performance when individual models capture different aspects of the underlying data distribution. On email data, the ensemble provides balanced performance, though Random Forest's bagging architecture (not shown in this comparison) proved even more effective for rich text. On SMS data, where feature sparsity challenges individual models, the ensemble's complementary strengths yield the best overall results. These visual comparisons confirm the ensemble learning principle that combining classifiers with complementary decision boundaries can achieve more robust predictions than any single model alone [10].

The comparison also highlights how the ensemble mitigates the weaknesses of its individual classifiers. SVM's conservative boundary produces very low False Positive Rates but unacceptably high False Negative Rates for security applications. XGBoost captures nonlinear feature relationships but may introduce slightly higher False Positive Rates. The ensemble tempers these extremes, achieving a more security-appropriate balance as evidenced by the bar heights across all metrics.

4.4 Cross-Platform Detection Implications

The cross-platform evaluation conducted in this study reveals that no single standalone classifier consistently dominates across different communication platforms.

As shown in Tables 9 and 10, Random Forest ranked highest on the email dataset, whereas the SVM+XGBoost ensemble ranked highest on the SMS dataset. This variation in performance highlights the impact of platform-specific text characteristics on model effectiveness.

Email messages typically contain longer textual content and richer contextual features, allowing tree-based models such as Random Forest to effectively capture complex feature interactions through their ensemble bagging architecture. The abundance of linguistic cues provides sufficient signal for decision trees to identify phishing indicators.

Conversely, SMS messages are significantly shorter and contain fewer contextual cues, resulting in sparse TF-IDF representations after vectorization. In such cases, margin-based classifiers like SVM provide strong regularization, while gradient boosting captures subtle patterns. The ensemble architecture evaluated in this study demonstrated the most balanced performance by combining these complementary approaches.

Thus, for institutions deploying phishing detection systems across multiple communication channels, these results suggest practical considerations.

First, platform-specific model selection may be necessary for optimal performance. A single model trained on email data may not generalize effectively to SMS messages.

Second, ensemble approaches offer a robust alternative when platform characteristics are mixed or when deployment across multiple channels is required. The SVM+XGBoost ensemble demonstrated strong performance on both datasets, suggesting good generalization capability.

Lastly, security requirements should guide model selection. Applications prioritizing minimal false alarms might favor SVM despite its higher false negative rate. Applications where missed attacks are catastrophic might prioritize recall-oriented models like XGBoost or Random Forest, or use threshold-tuned ensembles to balance both concerns.

The ensemble architecture evaluated in this study demonstrated the most balanced performance across both datasets, consistently achieving high accuracy and F1-scores without suffering from extreme trade-offs between precision and recall. By combining the strengths of SVM and XGBoost, the ensemble adapted more effectively to different message structures and feature distributions, supporting its viability for cross-platform phishing detection.

4.5 Alignment with Study Objectives

The cross-platform evaluation findings of this study directly address the research objectives outlined in Chapter 1.

Objective 1.3.2.1: The experimental evaluation successfully compared Random Forest, Support Vector Machine, and Neural Networks across both platforms. Results revealed clear performance differences depending on dataset characteristics, with Random Forest excelling on email and Neural Network ranking second on SMS.

Objective 1.3.2.2: XGBoost demonstrated competitive performance on both platforms, achieving high recall on email (0.9865) and reasonable F1-scores on SMS (0.9364). While not the top performer on either dataset, its consistent performance across platforms highlights its utility for cross-platform applications.

Objective 1.3.2.3: The proposed SVM+XGBoost weighted soft-voting ensemble was successfully implemented with probabilistic calibration. On the email dataset, the ensemble achieved competitive performance (F1-score of 0.9837), closely following Random Forest. On the SMS dataset, the ensemble achieved the highest overall performance (F1-score of 0.9575), demonstrating that hybrid approaches can effectively combine complementary learning strategies to improve detection capability, particularly on challenging short-text data.

Objective 1.3.2.4: The evaluation explicitly examined False Positive Rates and False Negative Rates alongside traditional metrics. The ensemble demonstrated the most balanced profile across both platforms, mitigating the extreme trade-offs observed in individual classifiers, particularly SVM's high precision but low recall, and XGBoost's stronger recall but higher false positive rate.

These results suggest that while platform-specific characteristics influence optimal model selection, ensemble approaches provide

more adaptable detection capability across heterogeneous phishing environments, supporting the primary motivation of this study.

5. CONCLUSION AND RECOMMENDATION

This cross-platform evaluation demonstrated that while Random Forest excels on email phishing detection (0.9864 accuracy, 0.0129 FNR), the proposed SVM+XGBoost ensemble provides the most balanced performance across platforms, achieving the highest accuracy on SMS (0.9841) and competitive results on email (0.9830). The ensemble successfully combines SVM's precision with XGBoost's recall, achieving competitive results on email and the highest performance on SMS.

Several areas for further research remain. First, future studies may explore larger and more diverse phishing datasets, including data collected from additional communication platforms such as social media messaging applications and web-based chat systems. This could further evaluate the generalization capability of machine learning models in real-world environments.

Second, future work may investigate more advanced ensemble strategies, such as stacking or hybrid meta-learning architectures, which could potentially provide improved performance compared to the weighted soft-voting approach used in this study.

Third, incorporating deep learning architectures such as transformer-based language models may enhance the ability of phishing detection systems to capture contextual relationships within textual data.

Finally, future research may consider real-time deployment considerations, including model latency, scalability, and integration with existing cybersecurity infrastructures.

Exploring these areas could further improve the effectiveness and practical applicability of machine learning-based phishing detection systems, contributing to more secure digital environments in educational institutions and beyond. These findings contribute to the development of more adaptable, security-aware phishing detection systems for educational institutions facing diverse communication threats.

6. REFERENCES

- [1] "Intelligent cybersecurity systems for phishing attack detection - An overview," *Computers & Electrical Engineering*, Nov. 2025.
- [2] H. D. Sepillo, G. M. Arcibal, B. J. Batan, G. B. Castronuevo, J. V. Estillore, and J. C. Mag-isa, "Phishing Attack Detection and Prevention in Educational Institutions Using Machine Learning: A Quantitative Experimental Design with Embedded Impact Analysis on Cybersecurity," *Cognizance Journal of Multidisciplinary Studies*, vol. 5, no. 10, pp. 403–410, Oct. 2025.
- [3] S. Latif and S. Pervaiz, "Detecting Phishing Attacks in Cybersecurity Using Machine Learning with Data

- Preprocessing and Feature Engineering," *Kashf Journal of Multidisciplinary Research*, Mar. 2025.
- [4] S. A. A. Ghaleb, "Machine Learning Approaches for Email Phishing Detection: A Comparative Study," *Journal of Cybersecurity and Information Management*, vol. 12, no. 2, pp. 45-58, 2023.
 - [5] M. Imani, A. Beikmohammadi, and H. R. Arabnia, "Comprehensive Analysis of Random Forest and XGBoost Performance with SMOTE, ADASYN, and GNUS Under Varying Imbalance Levels," *Technologies*, vol. 13, no. 3, art. 88, Feb. 2025.
 - [6] A. K. Jain and B. B. Gupta, "A Novel Approach for Phishing Email Detection Based on Ensemble Machine Learning," *International Journal of Information Security*, vol. 23, no. 4, pp. 567-582, Aug. 2024.
 - [7] P. K. K. Loh, A. Z. Y. Lee, and V. Balachandran, "Towards a Hybrid Security Framework for Phishing Awareness Education and Defense," *Future Internet*, vol. 16, no. 3, p. 86, Mar. 2024.
 - [8] N. A. Al-Am, "Phishing Email Dataset," *Kaggle*, May 2024. [Online]. Available: <https://www.kaggle.com/datasets/naserabdullahalam/phishing-g-email-dataset>. Accessed: Feb. 10, 2026.
 - [9] S. Mishra and D. Soni, "SMS Phishing Dataset for Machine Learning and Pattern Recognition," *Mendeley Data*, Jun. 2022. [Online]. Available: <https://data.mendeley.com/datasets/f45bkkt8pr/1>. Accessed: Feb. 10, 2026.
 - [10] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems*, 2000, pp. 1–15.
 - [11] R. Verma and N. Hossain, "Semantic feature selection for text with application to phishing email detection," in *Proc. IEEE Int. Conf. Cognitive Informatics*, 2012, pp. 455–460.
 - [12] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," *IEEE Access*, vol. 7, pp. 100363–100372, 2019.
 - [13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
 - [14] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, 1999, pp. 61–74.